COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	July 16,2026	**DATE SUBMITTED:**	July 16, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 2.1 — **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Screenshot

C++ Program Code:

```
#include <iostream>
#include <string>
#include <utility>

class Student {
    private:
        std::string studentName;
        int studentAge;
    public:
        // Constructor
        Student (std::string newName = "Cookie", int newAge = 7){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        // Destructor
        ~Student(){
            std::cout << "Destructor was called" << std::endl;
        }

        // Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

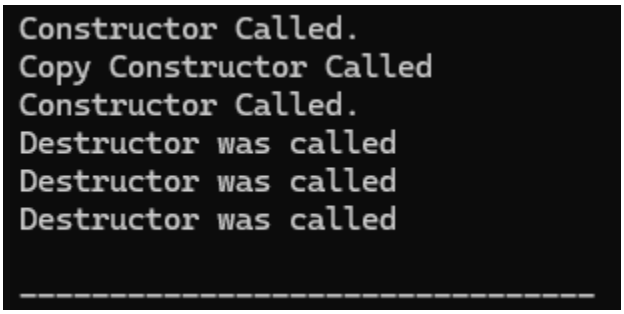
        // Display Attributes
        void printDetails(){
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }

};

int main() {

    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

Output	
Observation	The program starts off including three standard library headers as I am using (Dev C++): <iostream> for console input and output operations, <string> to utilize std::string class for text handling, and <utility> to be able to use utilities like std::move. The program first defines a class named Student, which enforces encapsulation by placing the studentName and studentAge variables in a private access scope. This lessens the risk of code errors as these member attributes under private cannot be altered or accessed directly by a normal external code. To be able to interact with this method, we also provide a public method which holds our constructor. This created constructor holds default parameters of a newName and newAge ("Cookie" and 7), with these initialized parameters, this constructor can still function as our default constructor if no arguments were assigned. The first line of the body of this constructor uses std::move(newName), it efficiently transfers the ownership. This is usually the most efficient and modern C++ pattern used, it is also called "sink" technique, wherein, the parameter acts as a TEMPORARY receiving area before the resources gets allocated or dropped into its final assignment. For duplication and Object cleanup we make the destructor and copy constructor.. The destructor frees the heap memory, closes files and/or releases database connections. In this code, the destructor executes automatic member cleanup. When the object Student goes out of scope it will release resources, example is it will print the "Destructor was called" to the console. The copy constructor is triggered whenever a new Student is initialized, as it is being initialized as a direct copy of an existing one. The function takes the source object by reference so it does not make a copy of it. This helps prevent errors in the code when passing or moving the arguments around. This way the source object isn't changed. Moreover, the class has a helper method called printDetails, this uses a pointer explicitly reference and is able to display the calling of the name and age attributes. The main() function serves as a guide for us to be able to track the exact sequence of the executions of constructors and destructors. First, student1 was initialized with arguments "Roman" and 28 ,which calls the parametrized constructor which then executes the custom that "Constructor was called.". Then, student2 is made using student1 as an initializer or reference, which then would call the copy constructor for duplication and will print the "Copy

	Constructor was called". Next, when student3 gets declared without arguments, it uses the default constructor arguments and prints the "Constructor was called." again. The next line student3 = student2 does not trigger any constructor as student3 already exists and has no need to be created; this can be considered by the computer as its default copy assignment operator. Lastly, all three objects are destroyed in reverse order of how they were created. This triggers another destructor one by one, which will then print "Destructor was called" being printed for another 3 times.

Table 2-1. Initial Driver Program

Screenshot	**C++ Program Code:**
	`#include <iostream>`
	`#include <string>`
	`#include <utility>`
	
	`class Student {`
	`private:`
	`std::string studentName;`
	`int studentAge;`
	`public:`
	`// Constructor`
	`Student (std::string newName = "Cookie", int newAge = 7){`
	`studentName = std::move(newName);`
	`studentAge = newAge;`
	`std::cout << "Constructor Called." << std::endl;`
	`};`
	`// Destructor`
	`~Student(){`
	`std::cout << "Destructor was called" << std::endl;`
	`}`
	
	`// Copy Constructor`
	`Student (const Student &copyStudent){`
	`std::cout << "Copy Constructor Called" << std::endl;`
	`studentName = copyStudent.studentName;`
	`studentAge = copyStudent.studentAge;`
	`}`
	
	`// Display Attributes`
	`void printDetails(){`
	`std::cout << this->studentName << " " << this->studentAge << std::endl;`
	`}`
	
	`};`

	``
	int main() {
	    const size_t j = 5;
	
	    Student studentList[j] = {};
	    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
	    int ageList[j] = {15, 16, 18, 19, 16};
	
	    return 0;
	}
	``

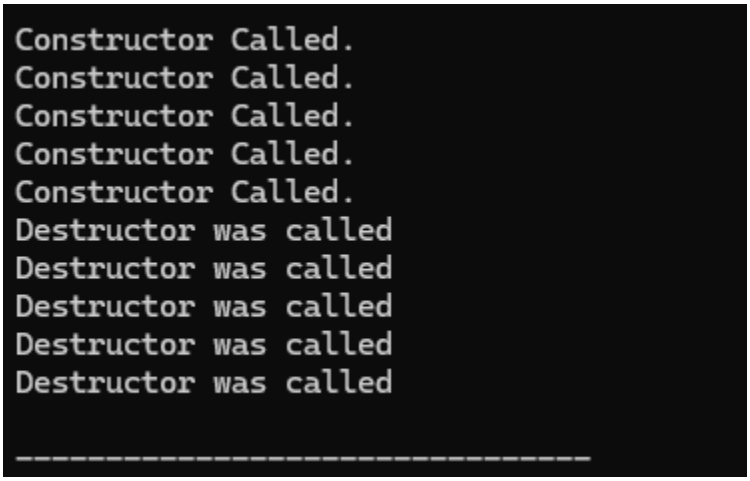
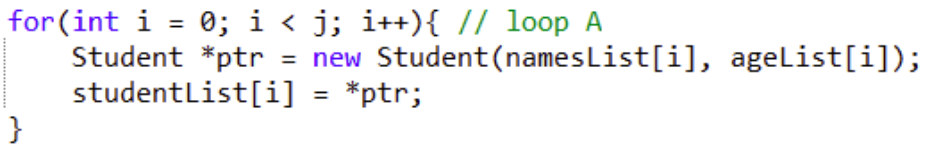
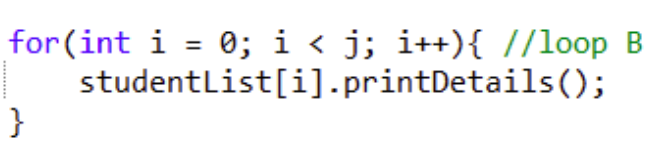
Output	
Observation	This code change gets rid of the variables in the main function and uses arrays with five spots instead. This way the program can handle students at the same time. The line const size_t j = 5 sets a size that is used to define the size of the arrays when the program is compiled. Then Student studentList[j] = {}; creates an array with five Student objects. The {} makes sure each object is set up properly with the default values. This means that each student is initially named Cookie and is 7 years old. The program also creates two arrays, namesList and ageList that have the same number of spots as the studentList array. These arrays can be used to store names and ages for the students. The program can use these arrays to update the studentList array,which we can use a loop to do it (in the final modification). The studentList array is where all the Student objects are stored. This change makes it easier for the program to manage students. The initiated arrays are lists of information that can be used to update the Student objects in the studentList array.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	Loop A runs five times to fill the array with the names and ages from your other array. Each time it runs it makes a Student object and stores it in the pointer. Then it copies the information from this object into the array. The problem is that this loop makes objects but never gets rid of the old ones so it wastes memory. According to research, this is called a memory leak. We can fix this by not using pointers at all. Instead we can just put the Student objects directly into the array. We can do this with the line of code: studentList[i] = Student(namesList[i] ageList[i]). This way is simpler. Do not waste any memory.
Loop B	
Observation	Loop B is where the program prints out or displays the information. It goes through the array one by one. For each student in the list it gets the Student object. Calls the printDetails function on it. This function prints out the name and age of each student. The program does this for all five students. Prints the information directly to the console. Loop B does this by going through the list from the student to the last one, which is at index j. 1. The studentList array has all the students in it and Loop B uses this array to print out the details of each Student object.

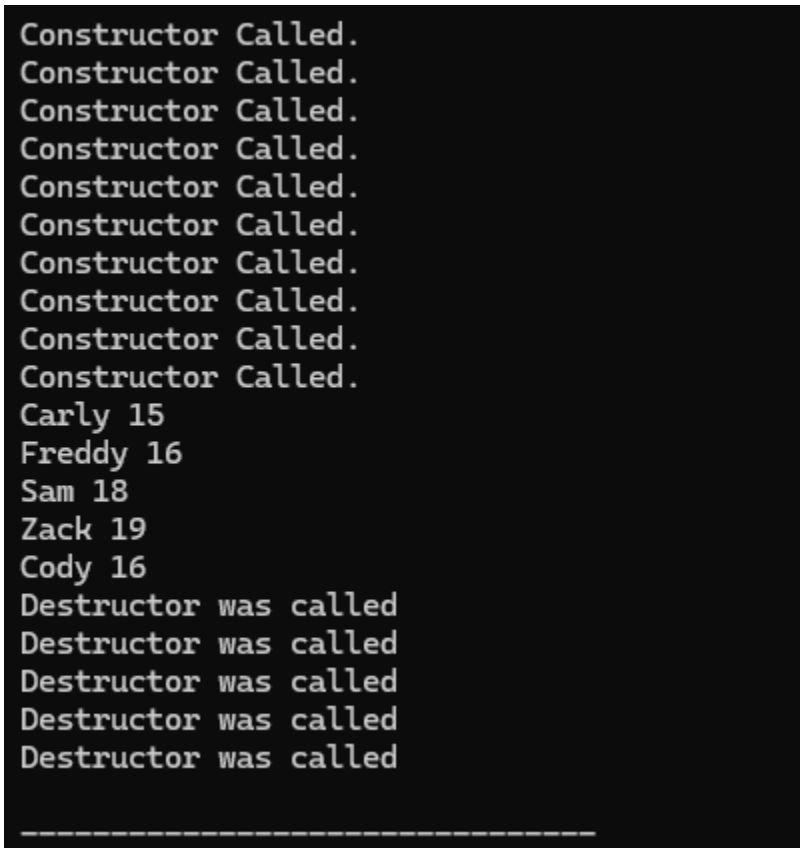
Output	
Observation	These two loops work together. Wherein, they first put information into the list of students made earlier. Then displays that information. Then proceeds with loop B that checks each of the students in the list and also prints out those details. Just like the explanation earlier, objects are destroyed in reverse order of how they were created. This triggers another destructor one by one, which will then print ”Destructor was called” being printed for another 3 times.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

C++ Program Code:

```cpp
#include <iostream>
#include <iomanip>
#include <string>

//==================== BASE CLASS ====================//

class GroceryItem {
protected:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    GroceryItem(std::string n = "", double p = 0, int q = 0)
        : name(n), price(p), quantity(q) {}

    // Copy Constructor
    GroceryItem(const GroceryItem& other)
        : name(other.name), price(other.price), quantity(other.quantity) {}

    // Copy Assignment Operator
    GroceryItem& operator=(const GroceryItem& other)
    {
        if (this != &other)
        {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    // Virtual Destructor
    virtual ~GroceryItem() {}

    double calculateSum() const
    {
        return price * quantity;
    }
```

```
    virtual std::string getType() const = 0;

    virtual void display() const
    {
        std::cout << std::left
                  << std::setw(12) << getType()
                  << std::setw(15) << name
                  << std::setw(10) << ("PHP " + std::to_string((int)price))
                  << std::setw(8) << ("x" + std::to_string(quantity))
                  << "PHP " << calculateSum()
                  << std::endl;
    }
};

//==================== FRUIT ====================//

class Fruit : public GroceryItem
{
public:
    Fruit(std::string n, double p, int q)
        : GroceryItem(n, p, q) {}

    Fruit(const Fruit& other)
        : GroceryItem(other) {}

    Fruit& operator=(const Fruit& other)
    {
        GroceryItem::operator=(other);
        return *this;
    }

    std::string getType() const override
    {
        return "Fruit";
    }

    ~Fruit() override
    {
        std::cout << "Fruit \"" << name << "\" deleted.\n";
    }
};
```

```cpp
//==================== VEGETABLE ====================//

class Vegetable : public GroceryItem
{
public:
    Vegetable(std::string n, double p, int q)
        : GroceryItem(n, p, q) {}

    Vegetable(const Vegetable& other)
        : GroceryItem(other) {}

    Vegetable& operator=(const Vegetable& other)
    {
        GroceryItem::operator=(other);
        return *this;
    }

    std::string getType() const override
    {
        return "Vegetable";
    }

    ~Vegetable() override
    {
        std::cout << "Vegetable \"" << name << "\" deleted.\n";
    }
};

//==================== TOTAL SUM ====================//

double TotalSum(GroceryItem* GroceryList[], int size)
{
    double total = 0;

    for (int i = 0; i < size; i++)
    {
        if (GroceryList[i] != nullptr)
            total += GroceryList[i]->calculateSum();
    }

    return total;
}
```

```cpp
//==================== DISPLAY ====================//

void DisplayList(GroceryItem* GroceryList[], int size)
{
    std::cout << "=================================================================\n";
    std::cout << "                    Jenna's Grocery List\n";
    std::cout << "=================================================================\n";

    std::cout << std::left
              << std::setw(12) << "Type"
              << std::setw(15) << "Item"
              << std::setw(10) << "Price"
              << std::setw(8) << "Qty"
              << "Subtotal\n";

    std::cout << "-----------------------------------------------------------------\n";

    for (int i = 0; i < size; i++)
    {
        if (GroceryList[i] != nullptr)
            GroceryList[i]->display();
    }

    std::cout << "-----------------------------------------------------------------\n";
    std::cout << "Total Amount: PHP " << TotalSum(GroceryList, size) << "\n";
    std::cout << "=================================================================\n\n";
}
```

```cpp
//==================== MAIN ====================//

int main()
{
    const int SIZE = 4;

    GroceryItem* GroceryList[SIZE];

    // Dynamic Memory Allocation
    GroceryList[0] = new Fruit("Apple", 10, 7);
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);

    std::cout << "ORIGINAL GROCERY LIST\n\n";
    DisplayList(GroceryList, SIZE);

    // Delete Lettuce
    std::cout << "Deleting Lettuce...\n\n";

    delete GroceryList[3];
    GroceryList[3] = nullptr;

    std::cout << "UPDATED GROCERY LIST\n\n";
    DisplayList(GroceryList, SIZE);

    // Memory Cleanup
    for (int i = 0; i < SIZE; i++)
    {
        delete GroceryList[i];
        GroceryList[i] = nullptr;
    }

    return 0;
}
```

Google Colab (For a better view, for presentation):
https://colab.research.google.com/drive/11UYpQ-m4xC7hXazZUMND4RKGWxPpWcav?usp=sharing

Output:

```
ORIGINAL GROCERY LIST

================================================================
                  Jenna's Grocery List
================================================================
Type        Item           Price     Qty      Subtotal
----------------------------------------------------------------
Fruit       Apple          PHP 10    x7       PHP 70
Fruit       Banana         PHP 10    x8       PHP 80
Vegetable   Broccoli       PHP 60    x12      PHP 720
Vegetable   Lettuce        PHP 50    x10      PHP 500
----------------------------------------------------------------
Total Amount: PHP 1370
================================================================

Deleting Lettuce...

Vegetable "Lettuce" deleted.
UPDATED GROCERY LIST

================================================================
                  Jenna's Grocery List
================================================================
Type        Item           Price     Qty      Subtotal
----------------------------------------------------------------
Fruit       Apple          PHP 10    x7       PHP 70
Fruit       Banana         PHP 10    x8       PHP 80
Vegetable   Broccoli       PHP 60    x12      PHP 720
----------------------------------------------------------------
Total Amount: PHP 870
================================================================

Fruit "Apple" deleted.
Fruit "Banana" deleted.
Vegetable "Broccoli" deleted.

--------------------------------
Process exited after 0.1056 seconds with return value 0
Press any key to continue . . .
```

In-Depth Code Analysis:

The program starts by including three header files: <iostream>, <iomanip>, and <string>. The <iostream> library is used to display output on the screen using std::cout. The <iomanip> library is used to organize the output into a table by setting the width and alignment of each column. The <string> library allows the program to use the std::string data type to store the names of the grocery items.

The GroceryItem class is the base class of the program. It contains the common information that every grocery item needs, which are the name, price, and quantity. These variables are declared as protected, which means they can be used by the classes that inherit from GroceryItem but cannot be accessed directly from outside the class. The constructor is used to assign values to these variables whenever a new object is created. The class also has a copy constructor and a copy assignment operator, which copy the values of one object to another. A virtual destructor is included so that objects created using dynamic memory can be deleted properly through a base class pointer.

The calculateSum() function is responsible for finding the subtotal of one grocery item. It simply multiplies the price by the quantity and returns the result. This function is used whenever the program needs the subtotal of an item, so the same calculation does not have to be written multiple times. The class also contains the display() function, which prints the type of item, its name, price, quantity, and subtotal in a table. The function uses std::setw() to keep each column aligned, making the output easier to read.

The getType() function is declared as a pure virtual function in the GroceryItem class. This means that the base class does not provide its own version of the function. Instead, each class that inherits from GroceryItem must define its own getType() function. This allows the program to display whether an item is a fruit or a vegetable.

The Fruit and Vegetable classes both inherit from the GroceryItem class. Since they share the same data members and most of the same functions, they only need to provide their own version of the getType() function. The Fruit class returns the word "Fruit", while the Vegetable class returns "Vegetable". Each class also has its own constructor, copy constructor, copy assignment operator, and destructor. The destructors display a message when an object is deleted, making it easy to see when memory is released.

The TotalSum() function calculates the total amount of all the grocery items. It receives the GroceryList array and its size as parameters. Using a for loop, the function goes through each element of the array. Before calculating

the subtotal, it checks if the pointer is not nullptr. If the pointer is valid, it calls the calculateSum() function and adds the returned value to the total. After checking all items, the function returns the final total amount.

The DisplayList() function is used to print the grocery list. It first prints the table title and the column headings. Then it uses a loop to go through every object stored in the GroceryList array. If an object exists, it calls the object's display() function to print its information. After all items have been displayed, the function calls TotalSum() to print the total amount. By placing the display code in its own function, the program becomes easier to read because the same code does not need to be written more than once.

The main() function controls the execution of the program. It first creates an array named GroceryList that stores pointers to GroceryItem objects. Four objects are then created using the new operator: two Fruit objects named Apple and Banana, and two Vegetable objects named Broccoli and Lettuce. These objects are stored in the same array, making it possible to manage all grocery items together. The program then calls DisplayList() to show the complete grocery list and the total amount.

After displaying the original list, the program removes the Lettuce object by calling the delete operator. The pointer is then set to nullptr to show that it no longer points to a valid object. The updated grocery list is displayed again, and the total amount is recalculated without the deleted item. At the end of the program, a loop deletes the remaining objects in the array to free the allocated memory before the program ends. This ensures that all dynamically allocated objects are properly removed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The code in the Laboratory Manual is not compatible with Dev C++ alone, libraries had to be added and some settings in Compiler Options had to be edited.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.